Git Command Deep-Dive

Simulation Academy — .NET Diploma (Group 3)
Assignment 01 — Git Mastery & Problem Solving
Student: Belal Fayez

1. Git Squash

Definition

دمج عدة Commits في Commit واحدة لتبسيط تاريخ التغييرات. لا يوجد أمر مستقل اسمه `git squash`؛ من الطرق الشائعة `git rebase -i` أو `git merge --squash`.

When and Why

قبل دمج Feature Branch لتحسين وضوح الـ History. **تنبيه:** العملية تعيد كتابة الـ Commit Hashes؛ تجنّبها على تاريخ مشترك دون تنسيق. وإذا كانت الـ Commits منشورة، فقد يلزم Push من نوع `--force-with-lease` بعد الاتفاق مع الفريق.

Example

أثناء تنفيذ Login API عملت أربع Commits لإضافة الـ Endpoint وتصحيح Validation وتعديل JWT وتحسين الكود. قبل فتح Pull Request تريد أن تظهر كميزة واحدة واضحة للمراجع.

```
git rebase -i HEAD~4
```

داخل المحرر اترك أول Commit بـ `pick` وغيّر الباقي إلى `squash` (أو `s`) ثم احفظ وعدّل رسالة الـ Commit النهائية.

```
pick   a1b2c3d Add login endpoint
squash b2c3d4e Fix validation
squash c3d4e5f Fix JWT handling
squash d4e5f6a Refactor login service
```

---

2. Git Merge vs Git Rebase

Definition

كلاهما يدمج تغييرات من Branch أخرى، لكن `merge` يحافظ على شكل التفرعات في التاريخ، بينما `rebase` يعيد تطبيق Commits الـ Feature فوق Base جديدة، وينتج عادةً تاريخًا خطيًا.

When and Why

استخدم `merge` حين يكون الحفاظ على التاريخ المشترك مهمًا، و`rebase` عندما تريد تاريخًا مرتبًا قبل الدمج. **تنبيه:** قد تحدث Conflicts في الحالتين، ولا تعمل Rebase لتاريخ يتعاون عليه الآخرون دون اتفاق.

Example

تعمل على `feature/booking-api` بينما تم تحديث `main`. تريد جلب آخر التغييرات قبل فتح Pull Request.

خيار Merge:

```
git switch feature/booking-api
git fetch origin
git merge origin/main
```

خيار Rebase:

```
git switch feature/booking-api
git fetch origin
git rebase origin/main
```

المقارنة	Merge	Rebase
التاريخ	يحتفظ بتاريخ التفرعات	بتسلسل جديد Commits يعيد تطبيق الـ
الـ Commit Hashes القديمة	لا يعيد كتابتها	المعاد تطبيقها Commits تتغير للـ
الاستخدام الشائع	دمج شغل مشترك بأمان نسبيًا	شخصية Feature Branch ترتيب تاريخ

3. Git Help

Definition

عرض توثيق Git الرسمي للأوامر والخيارات المتاحة.

When and Why

عند تعلّم أمر جديد أو التحقق من خيارات أمر حساس قبل تشغيله.

Example

تحتاج إلى اختيار Option مناسبة في `git log` ولا تتذكر الصيغة الصحيحة.

```
git help log
git log -h
git help -a
```

---

4. Git Cherry-Pick

Definition

تطبيق التغيير الذي أدخلته Commit محددة على الـ Branch الحالية، دون دمج جميع Commits الـ Branch المصدر.

When and Why

لنقل Bug Fix محدد أو تغيير صغير بين Branches. **تنبيه:** غالبًا ينشئ Commit جديدة بـ Hash مختلف، وقد تحدث Conflicts أو تنقصه تغييرات يعتمد عليها.

Example

يوجد إصلاح عاجل لخطأ في Authentication داخل Feature Branch، لكن باقي الـ Feature لم يكتمل. تريد نقل إصلاح الخطأ وحده إلى Branch مخصصة للإصلاح.

```
git switch -c hotfix/auth main
git cherry-pick a1b2c3d
```

---

5. Git Clean

Definition

حذف ملفات غير متتبعة (Untracked) من Working Directory. لا يحذف الملفات المتتبعة، ولا يحذف الملفات المتجاهلة افتراضيًا.

When and Why

قبل إعادة تشغيل Tests في بيئة نظيفة. **تنبيه:** قد يحذف ملفات محلية لا يمكن استرجاعها من Git، خصوصًا مع x-؛ راجع المعاينة أولًا.

Example

أثناء تجربة NET API. أنشأت ملفات Logs وتجارب مؤقتة غير متتبعة، وتريد تنظيف مجلد العمل.

```
git clean -n        # معاينة الملفات قبل الحذف
git clean -f        # حذف الملفات غير المتتبعة
git clean -fd       # حذف الملفات والمجلدات غير المتتبعة
git clean -ndx      # معاينة تشمل الملفات المتجاهلة
git clean -fdx      # حذف يشمل الملفات المتجاهلة أيضًا
```

---

6. Git Grep

Definition

البحث السريع عن نص أو Pattern داخل الملفات التي يتتبعها Git.

When and Why

أثناء Refactoring أو Debugging أو تتبع استخدام Class أو Method.

Example

تريد تغيير اسم `PaymentService` في مشروع .NET. وتحتاج معرفة كل المواضع التي تشير إليه.

```
git grep "PaymentService"
git grep -n "PaymentService"  # مع أرقام الأسطر
git grep -i "paymentservice"  # دون حساسية لحالة الحروف
git grep "PaymentService" HEAD  # البحث في نسخة آخر Commit
```

---

7. Git Blame

Definition

معرفة آخر Commit مسؤولة عن كل سطر في ملف، مع معلومات الكاتب والتاريخ.

When and Why

لفهم تاريخ سطر معين وربطه بالـ Requirement أو الـ Bug Fix السابق. **تنبيه:** يعرض آخر تعديل منسوب للسطر، وليس بالضرورة أول شخص كتبه، وليس أداة لإلقاء اللوم.

Example

وجدت شرطًا غير متوقع في `BookingService.cs` وتريد معرفة متى أُدخل وما السياق وراءه قبل تعديله.

```
git blame BookingService.cs
git blame -L 10,25 BookingService.cs
git show <commit-hash>
```

---

8. Git Bisect

Definition

استخدام Binary Search في تاريخ الـ Commits لتحديد التغيير الذي تسبب في Bug، بشرط معرفة نسخة سليمة وأخرى بها المشكلة.

When and Why

لتحديد Commit مسببة لمشكلة قابلة لإعادة الإنتاج دون فحص كل Commit يدويًا. **تنبيه:** احفظ تغييراتك المحلية قبل البدء، وقد تحتاج إلى `git bisect skip` لو تعذر اختبار نسخة معينة.

Example

Login API أصبحت ترجع 500 HTTP بعد عشرات الـ Commits. لا تعرف أي تعديل أدخل المشكلة.

```
git bisect start
git bisect bad                 # النسخة الحالية بها المشكلة
git bisect good a1b2c3d      # نسخة قديمة مؤكدة السلامة
# اختبر النسخة التي انتقل إليها Git، ثم:
git bisect good                # أو git bisect bad
# كرر الاختبار حتى تحديد الـ Commit
git bisect reset
```

---

9. Git Shortlog

Definition

تلخيص الـ Commit History وتجميع الـ Commits حسب الكاتب.

When and Why

لإعداد Release Notes أو مراجعة المساهمات. **تنبيه:** عدد الـ Commits ليس مقياسًا كافيًا لجودة أو إنتاجية المطور.

Example

تريد مراجعة قائمة المساهمين في إصدار جديد من المشروع وعدد Commits كل مساهم.

```
git shortlog -sn HEAD
git shortlog -sne HEAD
git shortlog --since="2026-09-01" HEAD
```

---

10. Git Prune

Definition

حذف Git Objects غير القابلة للوصول (Unreachable Objects) والتي انتهت مدة الاحتفاظ بها وفق إعدادات Git.

When and Why

لصيانة مستودع محلي عند الحاجة. **تنبيه:** لا تخلط بين `git prune` و`git fetch --prune`؛ الأخير يزيل مراجع Remote-Tracking القديمة. الحذف قد يصعّب استرجاع Commits مفقودة.

Example

بعد عمليات Rebase وReset متكررة، تريد إجراء صيانة للمستودع وتقليل البيانات القديمة غير المطلوبة.

```
git prune --dry-run   # معاينة فقط
git prune             # تنظيف وفق مدة الاحتفاظ الافتراضية
git gc                # الصيانة المعتادة الموصى بها غالبًا
```

---

11. Git Worktree

Definition

فتح أكثر من Working Directory مرتبطين بنفس Repository، بحيث تستطيع العمل على Branches مختلفة في وقت واحد.

When and Why

للعمل المتوازي على Feature وHotfix أو مقارنة إصدارات دون Clone إضافي. **تنبيه:** لا تزل Worktree فيها تغييرات غير محفوظة.

Example

تعمل على Feature الدفع ولديك تعديلات غير مكتملة، ثم يأتي Hotfix عاجل للـ Login. تريد العمل عليه دون تغيير الـ Working Directory الحالية.

```
git worktree add -b hotfix/login ../login-hotfix main
git worktree list
# بعد الانتهاء، ومن خارج مجلد الـ Worktree المراد حذفه:
git worktree remove ../login-hotfix
```

---

12. Git Verify-Commit

Definition

التحقق من التوقيع الرقمي الموجود على Commit، باستخدام إعدادات الثقة والمفاتيح المتاحة محليًا.

When and Why

للتحقق من سلامة التوقيع وهوية المفتاح وفق سياسة الفريق. **تنبيه:** نجاح التحقق لا يعني أن الكود آمن أو خالٍ من الأخطاء، ولا يمكن التحقق من Commit غير موقّعة بالطريقة نفسها.

Example

فريق العمل يشترط Signed Commits لتغييرات Authentication. تريد التحقق من توقيع Commit قبل مراجعتها.

```
git verify-commit HEAD
git verify-commit -v HEAD
git verify-commit a1b2c3d
```

بعد إعداد توقيع GPG أو SSH يمكن إنشاء Commit موقّعة:

```
git commit -S -m "Update authentication"
```

---

13. Git Filter-Repo

Definition

أداة خارجية (تُثبَّت بشكل مستقل) لإعادة كتابة Git History، مثل إزالة ملفات من جميع الـ Commits أو إعادة تنظيم مسارات قديمة.

When and Why

لإزالة ملفات حساسة أو كبيرة من تاريخ المستودع. **تنبيه:** العملية تغيّر Commit Hashes وقد تتطلب إعادة نشر التاريخ على الـ Remote ومعالجة الـ Forks والنسخ القديمة؛ إزالة الملف لا تضمن اختفاء الأسرار من كل النسخ.

Example

تم رفع ملف `.env` يحتوي على API Key بالخطأ إلى GitHub. حذفه في Commit جديدة لا يمحوه من التاريخ السابق.

أولًا: ألغِ المفتاح المكشوف أو غيّره (Rotate/Revoke)، ثم نسّق مع الفريق بشأن تنظيف التاريخ، ونفّذ العملية على Clone جديدة مخصصة لذلك:

```
git filter-repo --path .env --invert-paths
```

Quick Revision ##

الأمر	الهدف
Squash	دمج عدة Commits في واحدة
Merge	دمج التفرعات مع الحفاظ على تاريخها
Rebase	إعادة تطبيق Commits فوق Base جديدة
Help	قراءة توثيق الأوامر
Cherry-pick	نقل تغيير Commit معينة
Clean	حذف ملفات غير متتبعة
Grep	البحث في ملفات Git
Blame	تتبع آخر تعديل على سطر
Bisect	إيجاد Commit سببت Bug
Shortlog	تلخيص Commits حسب الكاتب
Prune	تنظيف Objects غير قابلة للوصول
Worktree	العمل على أكثر من Branch في مجلدات منفصلة
Verify-commit	التحقق من توقيع Commit
Filter-repo	إعادة كتابة التاريخ

مراجع رسمية للتوسع

(https://git-scm.com/docs)[Git Documentation] -
(https://git-scm.com/docs/git-rebase)[Interactive Rebase] -
(https://git-scm.com/docs/git-merge)[git merge] -
(https://git-scm.com/docs/git-cherry-pick)[git cherry-pick] -
(https://github.com/newren/git-filter-repo)[git filter-repo project] -

---

Quick Revision

Command	Purpose
Squash	Combine several commits into one
Merge	Integrate branches while preserving their history
Rebase	Replay commits on a new base
Help	Read Git documentation
Cherry-pick	Apply one selected commit
Clean	Remove untracked files
Grep	Search tracked content
Blame	Trace the latest change to each line
Bisect	Find the commit that introduced a bug
Shortlog	Summarize commits by author
Prune	Remove eligible unreachable objects
Worktree	Use separate working directories
Verify-commit	Verify a commit signature
Filter-repo	Rewrite repository history

References

- Git Documentation
- Interactive Rebase
- Git Merge
- Git Cherry-Pick
- Git Filter-Repo